



R3.04 – Qualité de développement

Le design pattern Builder

Réalisé par :
Pitavy Mathis
Priot Louise
Teixeira da Silva Samuel

IUT du Limousin

Sommaire

1. Les design patterns et leurs familles	3
2. Le problème des constructeurs télescopiques	3
3. Le principe du Builder	3
4. Les participants	4
5. Exemples d'utilisation	4
6. Avantages et inconvénients	4
7. Conclusion et références	5

1. Les design patterns et leurs familles

Un design pattern est une solution connue à un problème qui revient souvent lors de la conception d'un logiciel. Il sert de modèle pour organiser le code, mais il doit être adapté au problème rencontré.

On distingue trois familles principales :

- Les patrons de création, qui concernent la création des objets.
- Les patrons structurels, qui organisent les classes et les objets.
- Les patrons comportementaux, qui concernent les échanges et la répartition des responsabilités.

Le Builder fait partie des patrons de création. Il permet de construire un objet complexe en plusieurs étapes.

2. Le problème des constructeurs télescopiques

Quand une classe possède beaucoup d'attributs, certains obligatoires et d'autres facultatifs, on peut être amené à écrire plusieurs constructeurs. Cela devient vite difficile à lire : les appels contiennent beaucoup de paramètres et il n'est pas toujours évident de savoir à quel attribut correspond chaque valeur. On peut aussi se retrouver à utiliser de nombreuses valeurs null ou false.

Dans le support, l'exemple est celui d'un livre. Il peut avoir un titre, un auteur, un ISBN, un résumé, une maison d'édition et un prix, mais aussi des options comme le jaspé, le signet ou le rabat.

3. Le principe du Builder

Avec le Builder, l'objet est préparé étape par étape. On indique les caractéristiques dont on a besoin, puis on récupère l'objet construit. Cela évite de devoir passer toutes les valeurs dans un seul constructeur.

Cette méthode permet de créer plusieurs variantes d'un même type d'objet. Elle est pratique lorsque certains paramètres sont facultatifs ou lorsque les objets peuvent avoir différentes configurations. Un Director peut aussi être utilisé pour regrouper des étapes et proposer des configurations déjà préparées.

4. Les participants

Rôle	Responsabilité
Builder	Définit les étapes de construction.
ConcreteBuilder	Réalise les étapes, assemble l'objet et permet de récupérer le résultat.
Director	Organise les étapes en utilisant le Builder.
Product	L'objet qui est construit.

5. Exemples d'utilisation

Création d'un livre

Pour un logiciel de gestion de maison d'édition, un livre peut être défini par plusieurs informations (titre, auteur, ISBN, résumé, éditeur, prix) et par des options supplémentaires. Le Builder permet de choisir les éléments nécessaires sans avoir à renseigner toutes les options à chaque création.

Création d'un donjon

Le support présente aussi un exemple de jeu RPG. Un donjon peut avoir un nom, une difficulté, un boss, plusieurs étages, des pièges, des coffres et des monstres. Le Builder permet de préparer des donjons différents en configurant ces éléments.

6. Avantages et inconvénients

Les principaux avantages présentés sont :

- Le code est plus lisible, car les caractéristiques sont indiquées séparément.
- Les paramètres facultatifs sont plus faciles à gérer.
- Le pattern peut favoriser l'immuabilité de l'objet une fois construit.

Il présente aussi quelques limites :

- Il faut écrire davantage de code.
- La conception peut devenir plus complexe.
- Une mauvaise configuration peut conduire à un objet incomplet ou incorrect.

Le cours rappelle également les principes SOLID : responsabilité unique, ouvert/fermé, substitution de Liskov, ségrégation des interfaces et inversion des dépendances. Ce sont des principes généraux de conception à garder en tête lors du développement.

7. Conclusion et références

Le Builder est utile lorsqu'un objet possède beaucoup de caractéristiques ou plusieurs options facultatives. Il rend sa construction plus claire et permet de créer différentes configurations sans multiplier les constructeurs. En contrepartie, il ajoute des classes et du code : il est donc surtout intéressant lorsque la complexité de création le justifie.

Références du support

- Refactoring.Guru, « Builder » : <https://refactoring.guru/fr/design-patterns/builder>
- Yassine Chraa, « Le pattern Builder », Medium.
- GoF Pattern, « Builder Pattern » : <https://www.gofpattern.com/creational/patterns/builder-pattern.php>
- IONOS, « StringBuilder en Java ».
- Design Patterns: Elements of Reusable Object-Oriented Software.
- Wikipédia, « SOLID (informatique) ».